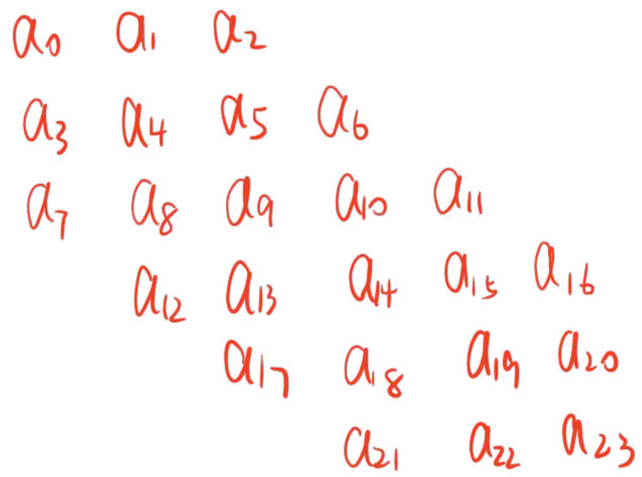
01 数据结构基础

- 数据结构概念
- 抽象数据类型
- 算法性能分析

02 线性表

- 线性表
- 数组
- 顺序表

作业



- 五对角矩阵中k与i、j的关系
- 考虑更一般的(2b+1)对角矩阵，设b ≤ n/2
- (1) 当0 ≤ i ≤ b-1时(上部区域)，第0行b+1个元素，第1行有b+2个元素，第i行有b+i+1个元素，因此第i行前共有i行，共有b+1 + b+2 + … + b+i = b*i + i*(i+1)/2 = (2b+i+1)*i/2，第i行第j列前有j个元素
- (2) 当b ≤ i ≤ n-b-1时(中部区域)，各行都有2b+1个元素，前b行(上部区域)共有(3b+1)*b/2个元素，在中部区域还有i-b行，有(i-b)*(2b+1)个元素，第i行第j列前有j-i+b个元素
- (3) 当n-b ≤ i ≤ n-1时(下部区域)，第n-b行有2b个元素，第n-b+1行有2b-1个元素，第n-1行有b+1个元素，前n-b行共有(3b+1)*b/2 + (n-2b)*(2b+1)个元素，除了上部和中部区域，还有i-n+b行，共有2b + 2b-1 + … + 2b-(i-n+b-1) = 2b*(i-n+b) – (i-n+b-1)*(i-n+b)/2 = (3b-i+n+1)*(i-n+b)/2个元素，第i行第j列前有j-i+b个元素

作业

- 编写一个程序，以不多于3n/2的平均比较次数，在一个有n个整数的顺序表中找出最大值和最小值的整数，要求使用的附加空间尽量少

```
void FindMaxMin(SeqList<int> &A, int &max, int &min) {
    int n = A.Length(), x;
    max = min = A.Get(0);
    for (int i = 1; i < n; i++) {
        x = A.Get(i);
        if (x > max) max = x;
        else if (x < min) min = x;
    }
}
```

- 最坏情况，n个整数从大到小排列，共比较2(n-1)次
- 最好情况，n个整数从小到大排列，共比较n-1次

作业

- 编写一个程序，从顺序表中删除具有给定值x的所有元素

```
template <class Type> void DelValue(SeqList<Type> &L, Type x) {
    if (L.IsEmpty())  { cerr<<"List  is empty!"<<endl; exit(1); }
    int len = L.Length();
    Type val;
    int i = 0;
    while (i < len) {
        val = L.Get(i);
        if (val == x) { L.Remove(i);  len--; }  //原地
        else i++;
    }
};
```

作业

- 编写一个程序，将顺序表中的元素进行原地逆置，即(a_0, a_1, …, a_{n-2}, a_{n-1})置换为(a_{n-1}, a_{n-2}, …, a_1, a_0)

```
template <class Type> int SeqList <Type> :: Reverse () {
    Type temp;
    for (int i = 0; i <= last/2; i++) {
        temp = list[i];
        list[i] = list[last-i];
        list[last-i] = temp;
    }
}
```

last = n-1;

a_0 a_1 a_2 a_3 a_4

02 线性表

- 链表
- 单链表
- 循环链表
- 双向链表
- 静态链表

作业

- 设有头结点的单链表，编程对表中任一值只保留一个结点，删除其余值相同的结点

```
template <class Type> void List<Type> :: DelDuplicate() {
    ListNode *p, *t, *pre;
    p = first->link; t = p;
    while (p != NULL) {
        pre = t; t = t->link;
        do {
            while (t != NULL && t->data != p->data) { pre = t; t = t->link; }
            if (t != NULL) { pre->link = t->link; delete t; t = pre->link; }
        } while (t != NULL);  // 找到所有重复的
        p = p->link; t = p;
    }
}
```

作业

- 编程实现“不保留原多项式，利用原多项式已有空间”的多项式链表相加操作

```
TermNode * PolyLinkedList :: PolyAdd(TermNode *ah, TermNode *bh) {
    TermNode *pa, *pb, *pc, *p;   //P指向删除的，PC指向生成多项式的表尾
    pc = pa = ah; p = pb = bh;  // 表头结点
    pa = pa->link; pb = pb->link;  // 第一个元素
    delete p;  // 删除bh的表头结点
    while ( pa != NULL && pb! = NULL ) {
        if ( pa->exp == pb->exp ) {
            pa->coef = pa->coef + pb->coef;  // 系数相加
            p = pb; pb = pb->link; delete p;
            if ( pa->coef == 0 ) { p = pa; pa = pa->link; delete p; }
            else { pc->link = pa; pc = pa; pa = pa->link; } //链入
        }
```

作业

- 编程实现“不保留原多项式，利用原多项式已有空间”的多项式链表相加操作

```
else if ( pa->exp < pb->exp) {
    pc->link = pa; pc = pa; pa = pa->link;
} else {
    pc->link = pb; pc = pb; pb = pb->link;
  }
}
if ( pa != NULL ) {
  pc->link = pa; //剩余整体链入
} else {
  pc->link = pb;
}
}
```

作业

- 设ha和hb分别是两个带表头结点的非递减有序单链表的表头指针，编程实现将这两个有序链表合并成一个非递增的有序单链表，要求使用原来两个链表的存储空间，不另外占用其它存储空间

```
template <class Type> void Merge (List<Type> &ha, List<Type> &hb) {
   ListNode<Type> *pa, *pb, *last, *q;
   pa = ha.First()->link; pb = hb.First()->link;  last = ha.First();  last->link = NULL;
   while (pa != NULL && pb != NULL) {
      if (pa->data <= pb->data) { q = pa; pa = pa->link; }
      else { q = pb; pb = pb->link; }
      q->link = last->link; last->link = q;  // 链入结果链表表头
   }                                  last = q;
   if (pb != NULL) pa = pb;
   while (pa != NULL) { q = pa; pa = pa->link; q->link = last->link; last->link = q; }
   q = hb.First(); delete q;                                          last = q;
};
```

作业

- **利用双向循环链表解决约瑟夫问题**

```
template <class Type> void Josephus (DblList <Type> & js, int n, int m) {
    DblNode <Type> *p = js.First()->rLink, *q;
    for ( int i = 1; i < n; i++ ) {  // 执行n-1次
        for ( int j = 1; j < m; j++ ) {  // 找到第m个结点
            if (p->rLink != js.First()) p = p->rLink;
            else p = js.First()->rLink; // 跳过附加表头结点
        }
        cout<<“出列的人是”<<p->data<<endl;
        p->lLink->rLink = p->rLink;
        p->rLink->lLink = p->lLink;
        q = p->rLink; delete p; p = q;
    }
};
```

03 串

- 串
- 串的模式匹配

作业

- 编写一个算法**frequency**，统计在一个输入字符串中各个不同字符出现的频度

```
void frequency(DString &s) {
    int LenOfChars = 0;  // 不相同字符个数
    int *freqs = new int[maxLen+1];
    char *temp = new char[maxLen+1];
    for (int j = 0; j < s.GetLen(); j++) {
        for (int k = 0; k < LenOfChars; k++) {
            if (temp[k] == s[j]) { freqs[k]++; break; } //已出现,计数加1
        }
        if (k == LenOfChars) { //未出现,加入
            temp[LenOfChars] = s[j]; freqs[LenOfChars++] = 1;
        }
    }
    // 打印结果，省略
}
```

作业

- **给定p=“abcabaa”与t=“abcaabbabcabaacbacba”，求解p的失效函数值，并给出按KMP算法进行匹配的过程**

j	0	1	2	3	4	5	6
p	a	b	c	a	b	a	a
next(j)	-1	0	0	0	1	2	1

第1趟 t a b c a a b b a b c a b a a c b a c b a
p a b c a b a a
第2趟 t a b c a a b b a b c a b a a c b a c b a
p a b c a b a a
第3趟 t a b c a a b b a b c a b a a c b a c b a
p a b c a b a a
第4趟 t a b c a a b b a b c a b a a c b a c b a
p a b c a b a a

作业

- **给定p=“abcabaa”与t=“abcaabbabcabaacbacba”，求解p的失效函数值，并给出按KMP算法进行匹配的过程**

j	0	1	2	3	4	5	6
p	a	b	c	a	b	a	a
next(j)	-1	0	0	0	1	2	1

第4趟 t a b c a a b b a b c a b a a c b a c b a

p a b c a b a a

第5趟 t a b c a a b b a b c a b a a c b a c b a

p a b c a b a a

04 栈和队列

- 栈
- 队列

作业

- n个元素依次进栈，则可能的出栈序列共有多少？
- 设1是第i+1个出栈的元素，则1前面的i个元素有b_i种出栈序列，而1后面的n-i-1个元素b_{n-i-1}种出栈序列，故有

$$b_n = \sum_{i=0}^{n-1} b_i \cdot b_{n-i-1}$$

$$b_n = \frac{1}{n+1} C_{2n}^{n} = \frac{1}{n+1} \frac{(2n)!}{n! \cdot n!}$$

作业

- 写出以下表达式的后缀表达式。
  - a*(b+c)-d
  - (a+a*b)*a+c*b/a
  - a+((b*c-d)/e+f*g/h)+i/j

- abc+*d-
- aab*+a*cb*a/+
- abc*d-e/fg*h/++ij/+

作业

- **利用栈实现单链表上的逆置运算。**

```
template <class Type> void reverse(List<Type> &L) {
    Stack<LinkNode<Type> *> S;
    LinkNode<Type> *p, *r;
    p = L.First()->link;
    while (p != NULL) {
        r = p->link; L.First()->link = r;  // 摘下p
        S.push(p);  // 入栈p
        p = r;
    }
    r = L.First();
    while (!S.IsEmpty()) {
        S.Pop(p); r->link = p; r = p;
    }
};
```

作业

- 设用链表表示一个双向队列，要求可以在表的两端插入，但只能在表的一端删除，试编写该队列的入队、出队、及判队空的算法。
- 采用不带附加表头结点的双向非循环链表作为存储表示

```
template <class Type> class DoubleQueue {
   private:
      DoubleQueueNode<Type> *end1, *end2;
   public:
      DoubleQueue () { end1 = NULL; end2 = NULL; }
      void EnQueue1 (Type x);  // 进队
      void EnQueue2 (Type x);  // 进队
      void DeQueue1 (Type& x);  // 出队
      bool IsEmpty() { end1 == NULL; }
};
```

作业

- 设用链表表示一个双向队列，要求可以在表的两端插入，但只能在表的一端删除，试编写该队列的入队、出队、及判队空的算法。
- 采用不带附加表头结点的双向非循环链表作为存储表示

```
template <class Type> void DoubleQueue<Type> :: EnQueue1 (Type x) {
    if (end1 == NULL)
        end2 = end1 = new DoubleQueueNode<Type>(x);
    else {
        end1->lLink = new DoubleQueueNode<Type>(x, NULL, end1);
    }
    end1 = end1->lLink;
}
```

左 右

作业

- 设用链表表示一个双向队列，要求可以在表的两端插入，但只能在表的一端删除，试编写该队列的入队、出队、及判队空的算法。
- 采用不带附加表头结点的双向非循环链表作为存储表示

```
template <class Type> void DoubleQueue<Type> :: EnQueue2 (Type x) {
    if (end2 == NULL)
        end2 = end1 = new DoubleQueueNode<Type>(x);
    else {
        end2->rLink = new DoubleQueueNode<Type>(x, end2, NULL);
    }
    end2 = end2->rLink;
}
```

作业

- 设用链表表示一个双向队列，要求可以在表的两端插入，但只能在表的一端删除，试编写该队列的入队、出队、及判队空的算法。
- 采用不带附加表头结点的双向非循环链表作为存储表示

```
template <class Type> void DoubleQueue<Type> :: DeQueue1 (Type &x) {
    if (IsEmpty()) { cerr<<“Queue is empty!”<<endl; exit(1); }
    DoubleQueueNode<Type> *p = end1;
    end1 = p->rLink;
    x = p->data;
    delete p;
}
```

05 递归和广义表

- 递归
- 递归转化为非递归
- 广义表

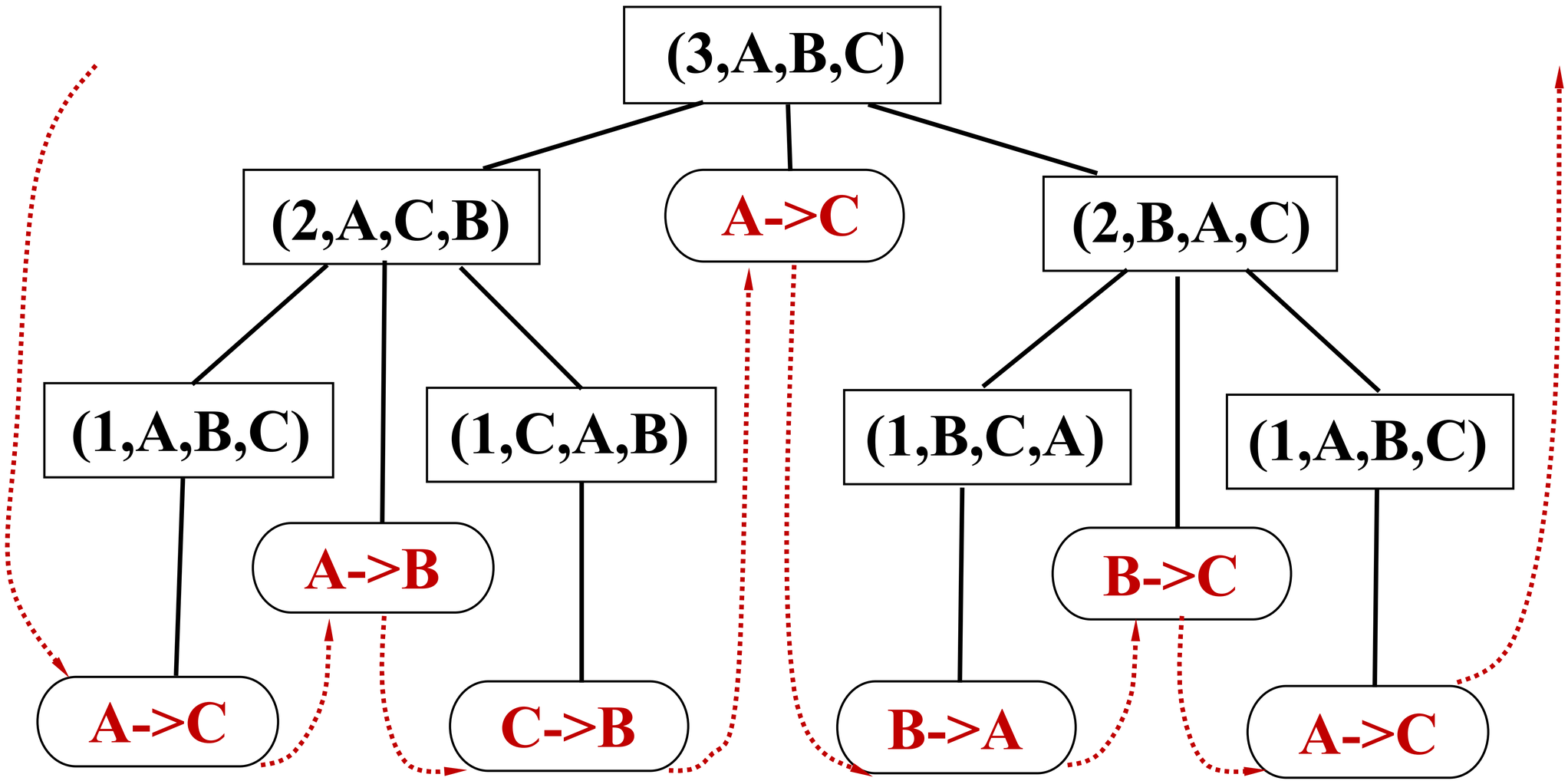
作业

- 实现迷宫问题的递归算法

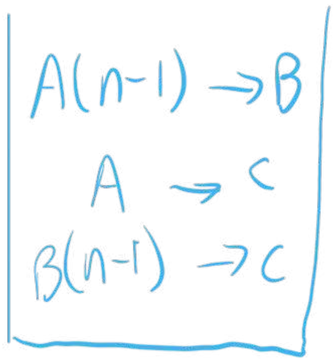
```
int SeekPath(int x, int y) {  // 寻找通向出口[m][p]的路径，出发点为[1][1]
    int i, g, h, d;  // g, h记录位置，d记录方向
    if (x == m && y == p)  return 1;  // 已达出口
    for (i = 0; i < 8; i++) {  // 依次按每一个方向寻找通向出口的路径
        g = x + move[i].a;  h = y + move[i].b;  d = i;  // 下一个位置
        if (Maze[g][h] == 0 && mark[g][h] == 0) {  // 下一个位置可通
            mark[g][h]=1;
            if (SeekPath(g, h)) {  // 从此位置递归试探
                cout<<"("<<g<<","<<h<<"),"<<"Direction" <<d<<",";
                return 1;
            }
        }
    }
    if (x == 1 && y == 1) { cout<<"No Path in Maze"<<endl;  return 0; }
}
```

作业

- 实现Hanoi塔问题的非递归算法



作业



- **实现Hanoi塔问题的非递归算法**

```
void towers(int n, int A, int B, int C) {
    Stack s;  int done = false;
    while (!done) {
        while (n>1) {
            s_tack(n, A, B, C, &s);  // 把它的参数加入栈中
            setvar1(&n, &A, &B, &C);  // 把n设为n-1，并且交换C和B的值
        }
        printf("\n %d -> %d\n", A, C);
        if (!empty(&s)) {
            restore(&n, &A, &B, &C, &s);  //恢复局部变量
            printf("\n %d -> %d\n", A, C);
            setvar2(&n, &A, &B, &C);  // 把n设为n-1，并且交换A和B的值
        } else done = true;
    }
}
```

作业

- 实现n皇后问题的非递归算法

```
void NQueens(int n) {
    int k = 0; q[k] = -1;  // k表示行数，  q[k]表示第k行皇后所在的列数
    while(k >= 0) {
        q[k] = q[k] + 1; //往前找
        while(q[k] < n && !Place(k))
            q[k] = q[k] + 1;  // 如果当前列不满足情况，则判断下一列
        if(q[k] < n) {  // 找到了可以放置的位置
            if(k < n-1) { k = k + 1; q[k] = -1; }
            else printf(q[0:n-1]);
        } else k = k - 1;  // 退回上一行重新选择满足条件的的列(回溯)
    }
}
int Place(int k) {
    for (int i = 0; i < k; i++) if(q[i] == q[k] || abs(q[i] - q[k]) == abs(i - k)) return 0;
    return 1;
}
```



作业

- **已知Ackerman函数定义，实现其递归与非递归算法**

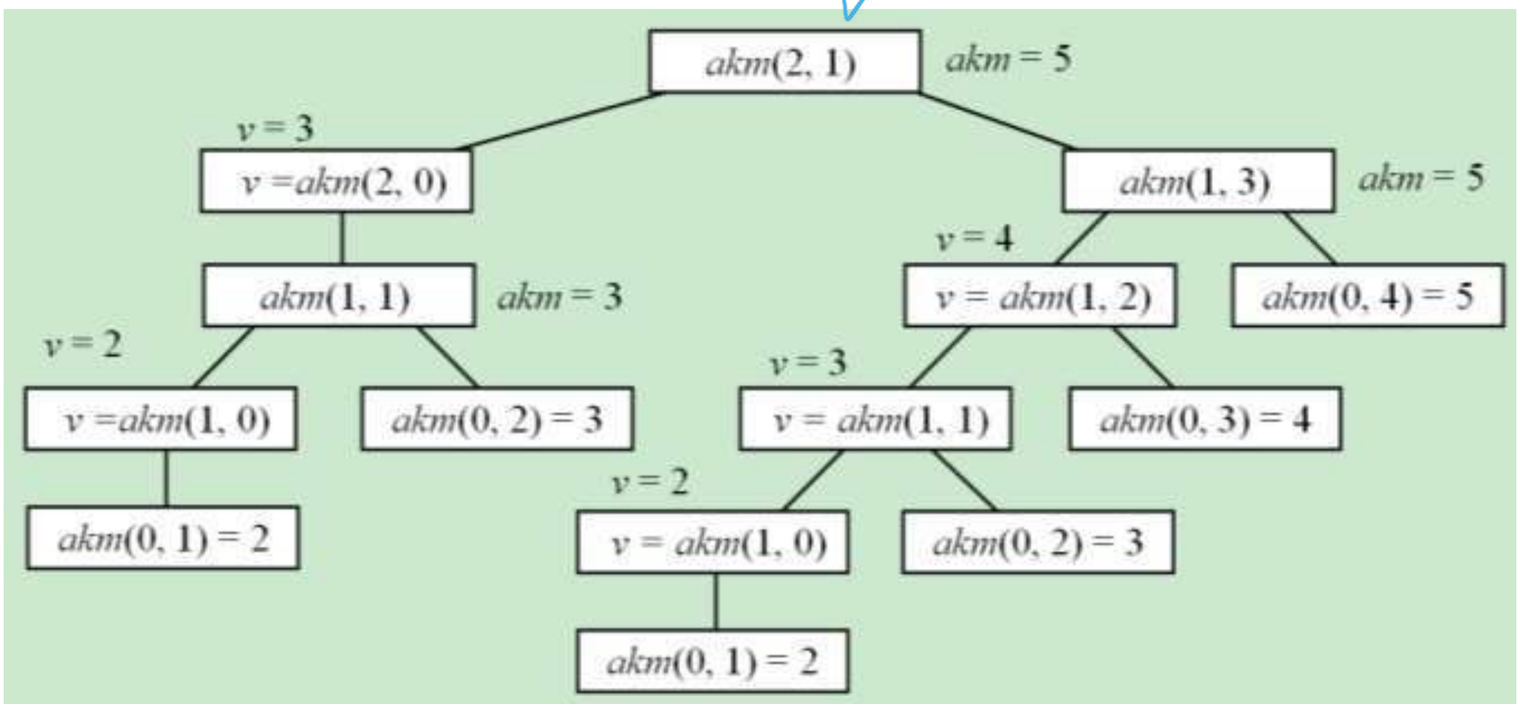
$$akm(m, n) = \begin{cases} n+1, & m = 0 \\ akm(m-1, 1), & m \neq 0, n = 0 \\ akm(m-1, akm(m, n-1)), & m \neq 0, n \neq 0 \end{cases}$$

```
int akm (int m, int n) {
    if (m == 0) return n+1;  // m == 0
    else if (n == 0) return akm(m-1, 1);  // m > 0, n == 0
    else return akm(m-1, akm(m, n-1));  // m > 0, n > 0
}
```

作业

- **已知Ackerman函数定义，实现其递归与非递归算法**

$$akm(m, n) = \begin{cases} n+1, & m = 0 \\ akm(m-1, 1), & m \neq 0, n = 0 \\ akm(m-1, akm(m, n-1)), & m \neq 0, n \neq 0 \end{cases}$$



作业

- **已知Ackerman函数定义，实现其递归与非递归算法**

$$akm(m,\ n) = \begin{cases} n+1, & m = 0 \\ akm(m-1,\ 1), & m \neq 0,\ n = 0 \\ akm(m-1,\ akm(m,\ n-1)), & m \neq 0,\ n \neq 0 \end{cases}$$

						1	0	0	1		
		2	0	1	1	0	?	0	?	0	2		
2	1	1	?	1	?	1	?	1	?	1	?	1	3

				1	0	0	1				
		1	1	0	?	0	?	0	2		
1	2	0	?	0	?	0	?	0	?	0	3		
0	?	0	?	0	?	0	?	0	?	0	?	0	4

作业

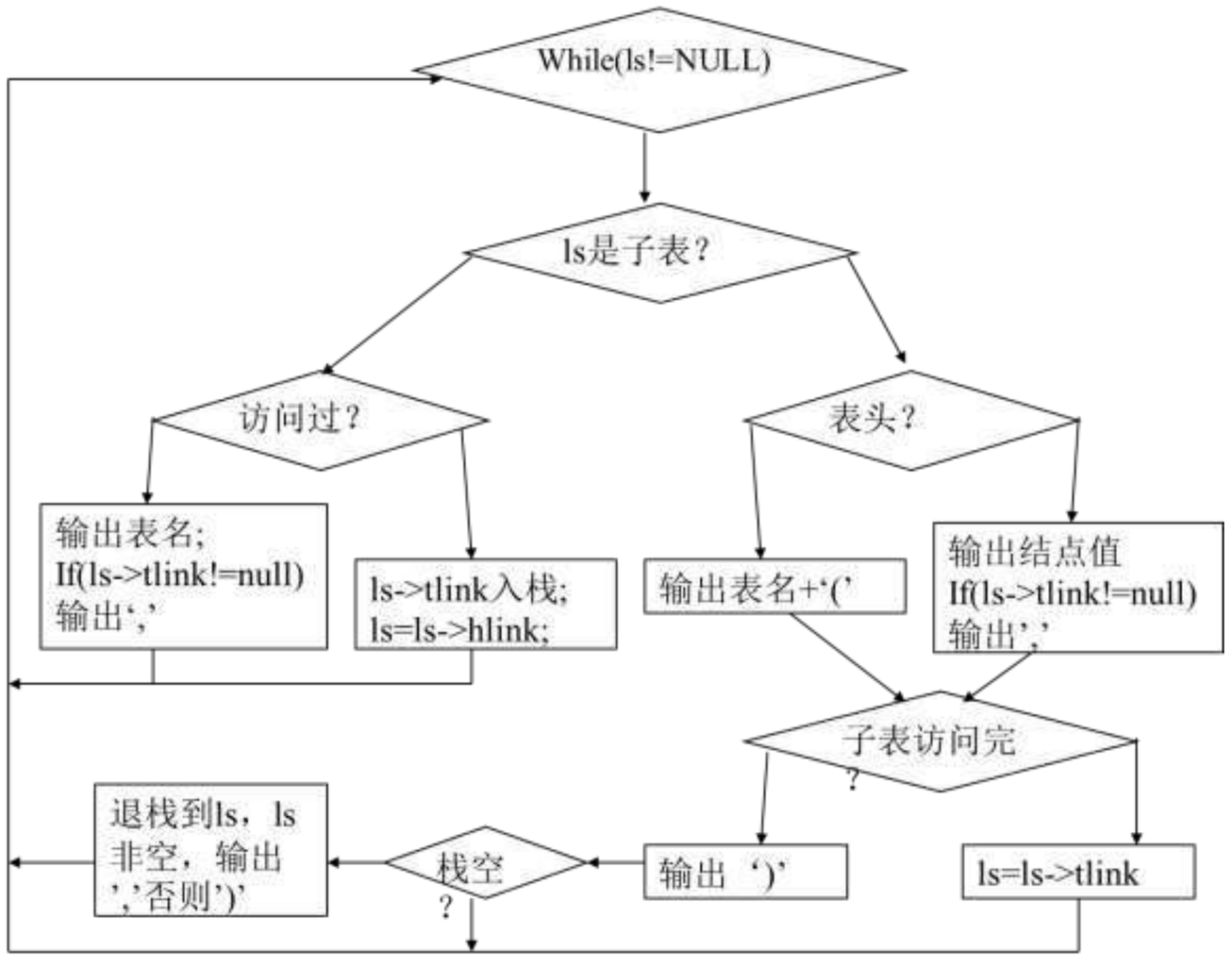
- **已知Ackerman函数定义，实现其递归与非递归算法**

$$akm(m, n) = \begin{cases} n+1, & m = 0 \\ akm(m-1, 1), & m \neq 0, n = 0 \\ akm(m-1, akm(m, n-1)), & m \neq 0, n \neq 0 \end{cases}$$

```
int akm (int m, int n) {
    int i, j, top = -1; int Sm[10], Sn[10];
    Sm[++top] = m; Sn[top] = n;
    while(1) {
        i = Sm[top]; j = Sn[top]; top--;
        if (i == 0)  if (top != -1)  Sn[top] = j+1; else return j+1;
        else if (j == 0) { Sm[++top] = i-1; Sn[top] = 1; }
        else { Sm[++top] = i-1; Sm[++top] = i; Sn[top] = j-1; }
    }
}
```

作业

- 实现广义表遍历的非递归算法



作业

- 实现广义表遍历的非递归算法

```
void GenList :: traverse ( GenListNode *ls ) {
   Stack <GenListNode<Type> *> st;
   while ( ls != NULL ) {
      ls->mark = 1;
   if ( ls->utype == 2 ) {  // 子表结点
      // 子表未访问过，暂存下一结点地址, 访问子表
      if ( ls->value.hlink->mark == 0 ) { st.Push( ls->tlink ); ls = ls->value.hlink; }
      else {
         cout << ls->value.hlink->value.listname;  // 该子表已访问过, 仅输出表名
         if ( ls->tlink != NULL ) { cout << ‘,’; ls = ls->tlink; }
      }
 }
```

作业

- 实现广义表遍历的非递归算法

```
else {  // 非子表结点
    if ( ls->utype == 0 )  cout << ls->value.listname << '(';  // 表头结点
    else if ( ls->utype == 1 ) {  // 原子结点
        cout << ls->value.atom;
        if ( ls->tlink != NULL ) cout <<',';
    }
    if ( ls->tlink == NULL ) {  // 子表访问完, 子表结束处理
        cout << ')';
        if ( !st.IsEmpty( ) ) {  // 栈不空
            st.Pop (ls);   // 退栈
            if ( ls != NULL ) cout << ',';  else cout << ')';
        }
    } else ls = ls->tlink;  // 向表尾搜索
  }
}
};
```

//结尾大括号

06 树

- 树
- 二叉树

作业

- **若用二叉链表作为二叉树的存储表示，编写在二叉树中查找值为x的结点并打印该结点所有祖先结点的算法(假设值为x的结点不多于1个)**

```
template <class Type> int Find(BinTreeNode<Type> *&BT,
  Type x, Type path[], int level,  int &count) {
   if (BT != NULL) {
       level++; path[level] = BT->data;  //level表示层次
       if (BT->data == x) { count = level; return 1; }
       if (Find(BT->leftChild, x, path, level, count)) return 1;
       return Find(BT->rightChild, x, path, level, count);
   } else return 0;
};
```

作业

- 若用二叉链表作为二叉树的存储表示，编写根据输入的广义表建立二叉树的算法
  - 广义表的表名放在表前，表示树的根结点，括号中是根的左、右子树。每个结点的左、右子树用逗号隔开。若仅有右子树没有左子树，逗号不能省略。在整个广义表表示输入的结尾加上一个特殊符号#，表示输入结束。例如，A(B(D, E(G,)), C(, F))#
- 依次从保存广义表的字符串ls中输入字符
  - 若是字母，则表示结点值，建立新结点，作为左子女(k=1)或右子女(k=2)链接到其父结点上
  - 若是(，则表示子表开始，将k置为1；若是)，则表明子表结束
  - 若是,，则表示以左子女为根的子树处理完毕，应接着处理以右子女为根的子树，将k置为2
  - 如此处理，直至读入结束符#为止
  - 使用一个栈，在进入子表之前将根结点指针进栈，以便括号内的子女链接之用，在子表处理结束时退栈

作业

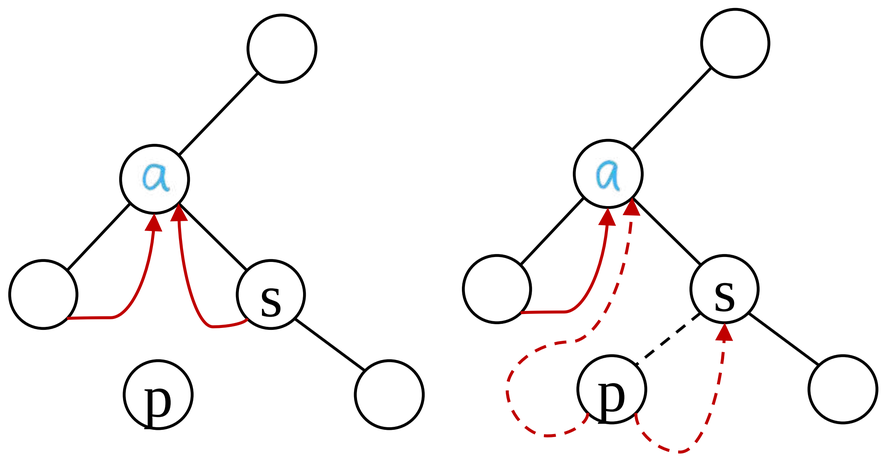
```
// 从输入流in输入二叉树的广义表表示建立对应的二叉链表
void CreateBinTree(istream &in, BinTreeNode <char> *&BT) {
    Stack <BinTreeNode <char> *> s;  BT = NULL;
    BinTreeNode <char> *p, *t;  int k;  char ch;
    in>>ch;
    while (ch != '#') {
        switch (ch) {
            case '(' : s.Push(p);  k = 1;  break;
            case ')' : s.Pop(t);  break;
            case ',' : k = 2;  break;
            default : p = new BinTreeNode(ch);
                      if (BT == NULL)  BT = p;
                      else if (k == 1) { s.GetTop(t);  t->leftChild = p; }
                      else { s.GetTop(t);  t->rightChild = p; }
        }
        in>>ch;
    }
}
```

06 树

- 二叉树的计数
- 线索化二叉树
- 堆
- 霍夫曼树

作业

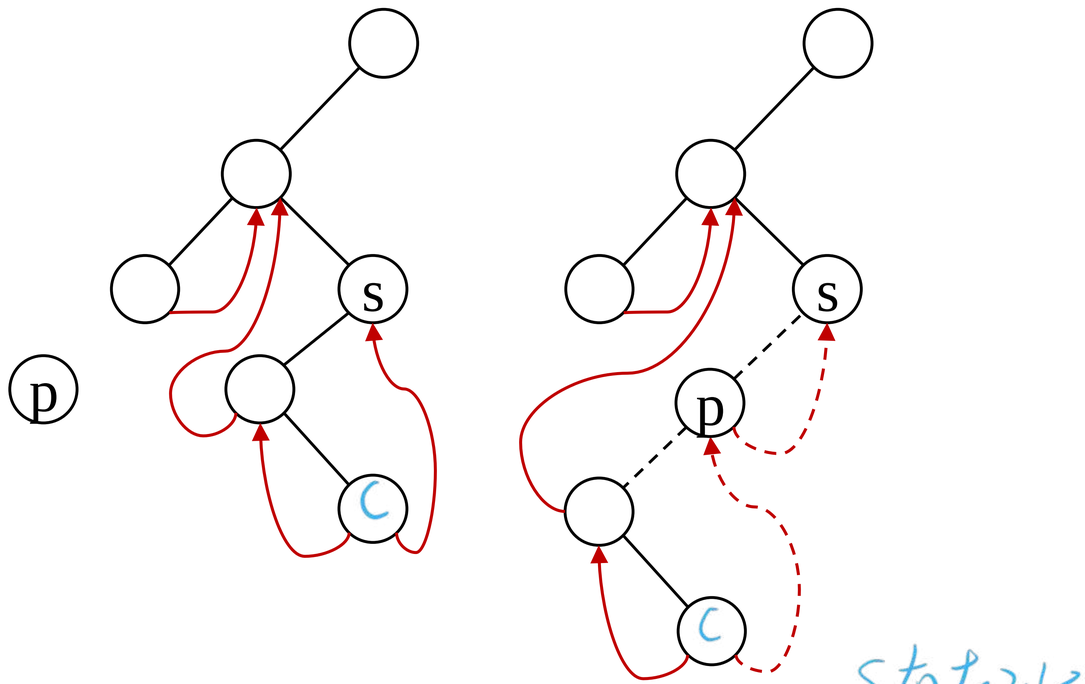
- 试编写一个算法，把一个新结点p作为结点s的左子女插入到中序线索化二叉树，s原来的左子女变成p的左子女



插入前后

s 无 左子女

as → a p s



插入前后

s 有左子女

(s → (p s

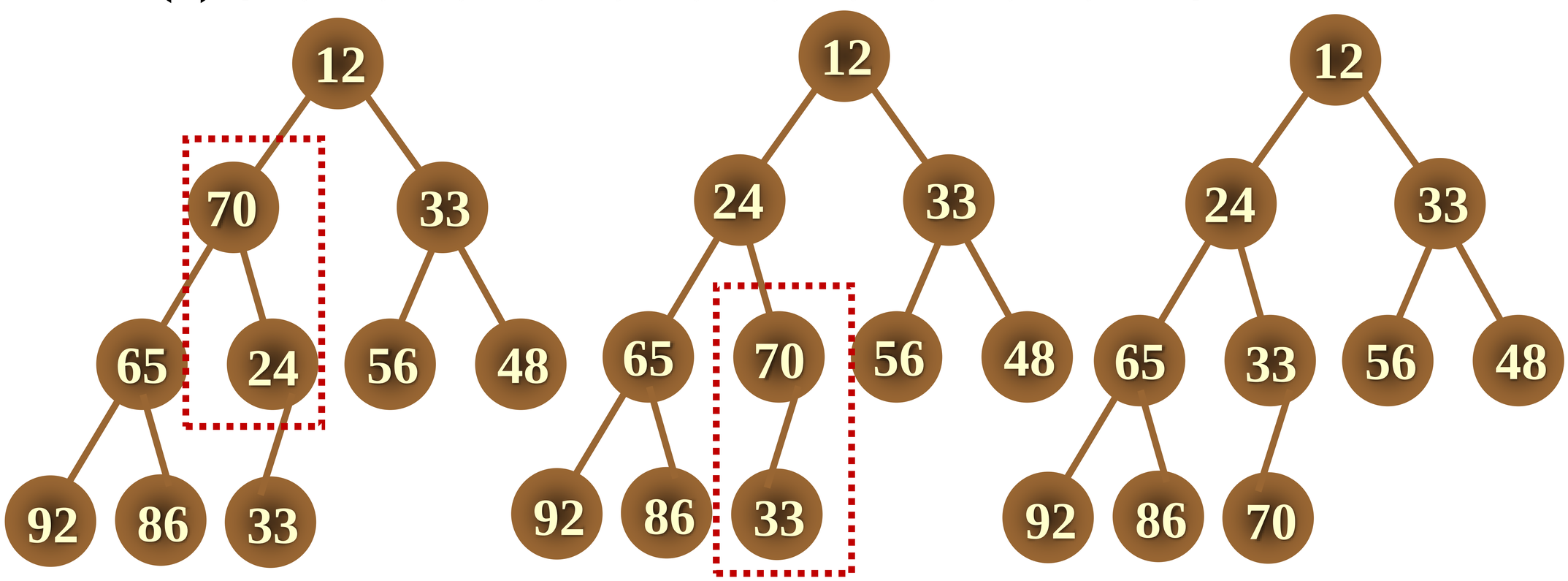
作业

- 试编写一个算法，把一个新结点p作为结点s的左子女插入到中序线索化二叉树，s原来的左子女变成p的左子女

```
template <class T> void InsertLeft(ThreadNode <T> *s, ThreadNode <T> *p) {
    // s的左子女指针或前驱线索传给p
    p->leftChild = s->leftChild; p->ltag = s->ltag;
    // p的rightChild成为指向s的后继线索
    p->rightChild = s;  p->rtag = 1;
    // p成为s的左子女
    s->leftChild = p;  s->ltag = 0;
    if (p->ltag == 0) {  // s原来有左子女
        // 在s原来的左子树中找中序下最后一个结点
        ThreadNode <Type> *temp = Last(p->leftChild);
        temp->rightChild = p;  // 建立指向p的后继线索
    }
}
```

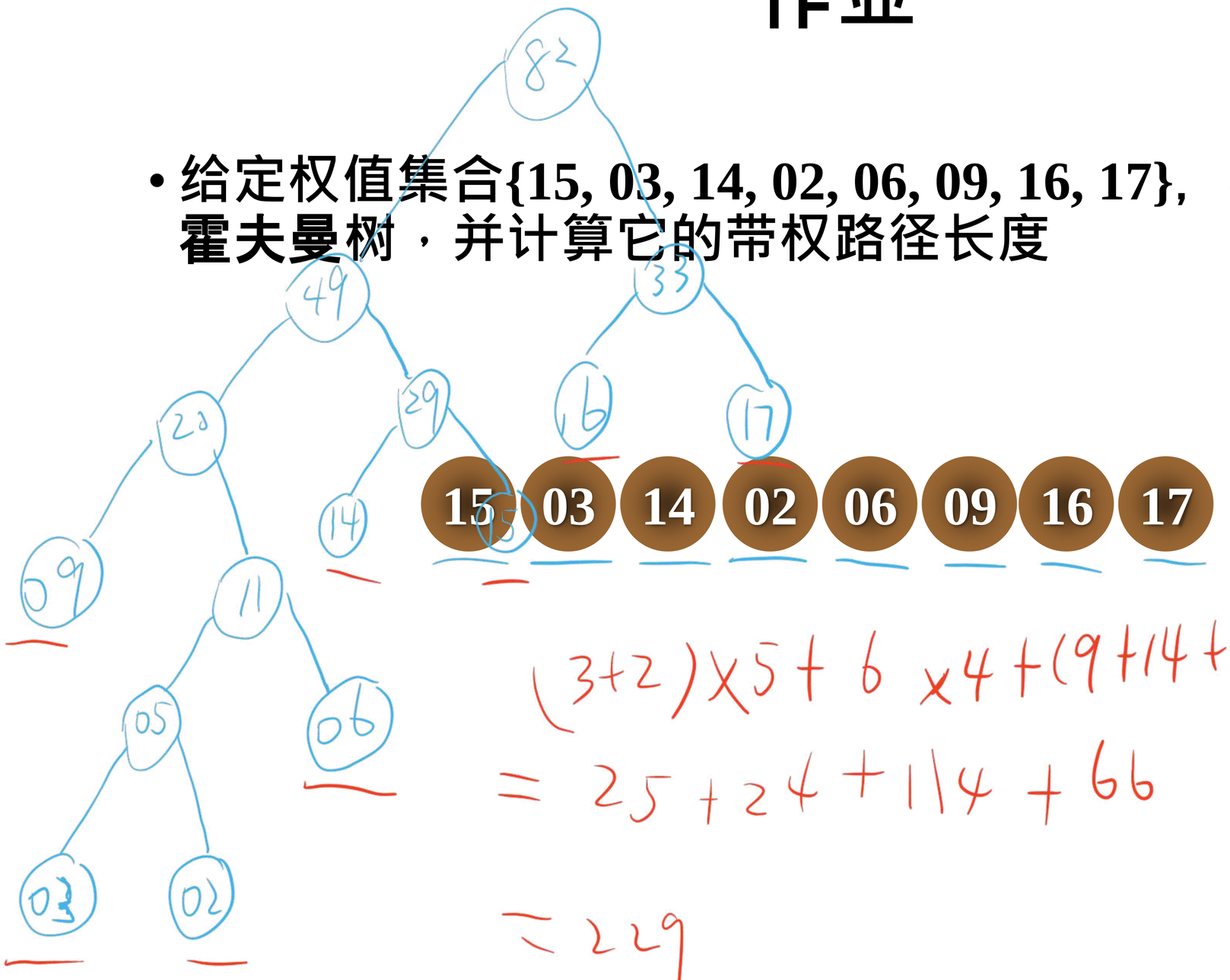
作业

- **判断以下序列是否是最小堆？如果不是，调整为最小堆**
- **(1) {12, 70, 33, 65, 24, 56, 48, 92, 86, 33}**
- **(2) {103, 97, 56, 38, 66, 23, 42, 12, 30, 52, 06, 20}**
- **(3) {05, 56, 20, 23, 40, 38, 29, 61, 35, 76, 28, 100}**



作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度



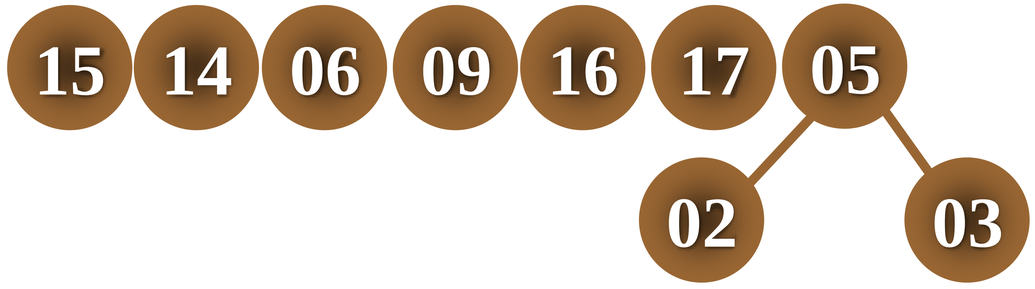
(3+2)×5+6×4+(9+14+15)×3+(16+17)×2

= 25+24+114+66

= 229

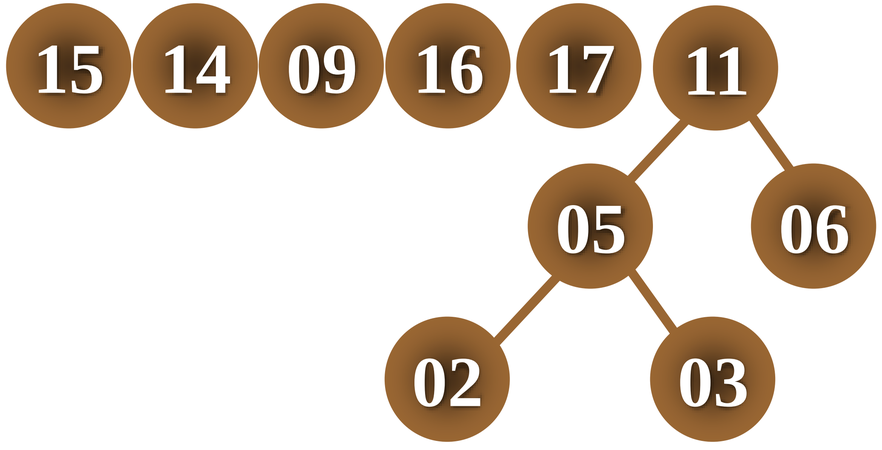
作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度



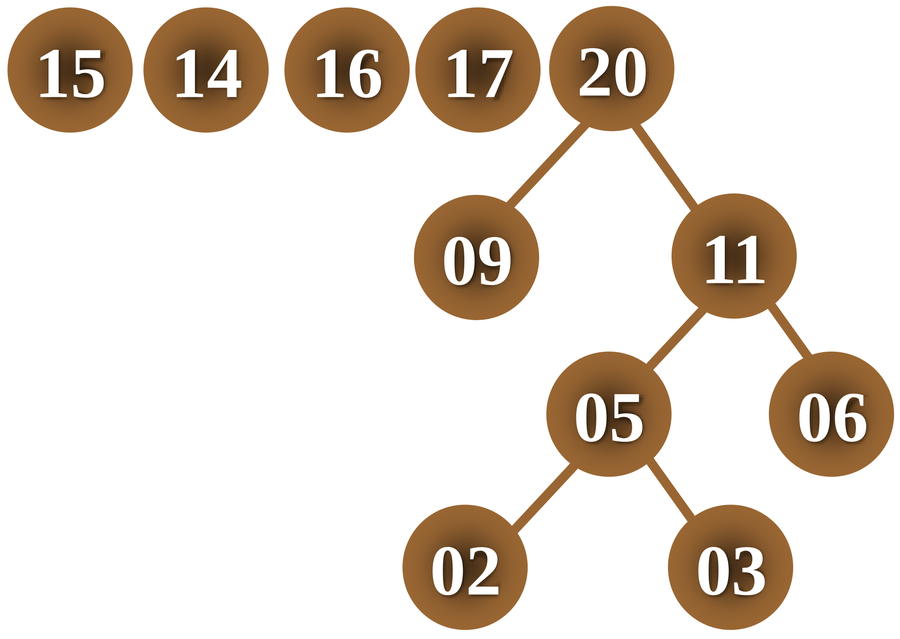
作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度



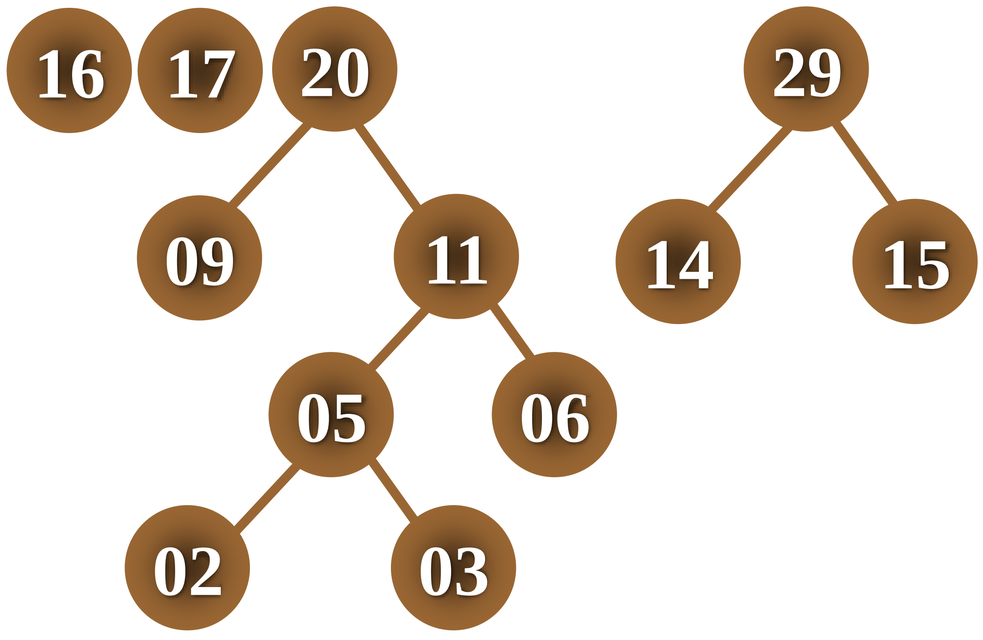
作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度



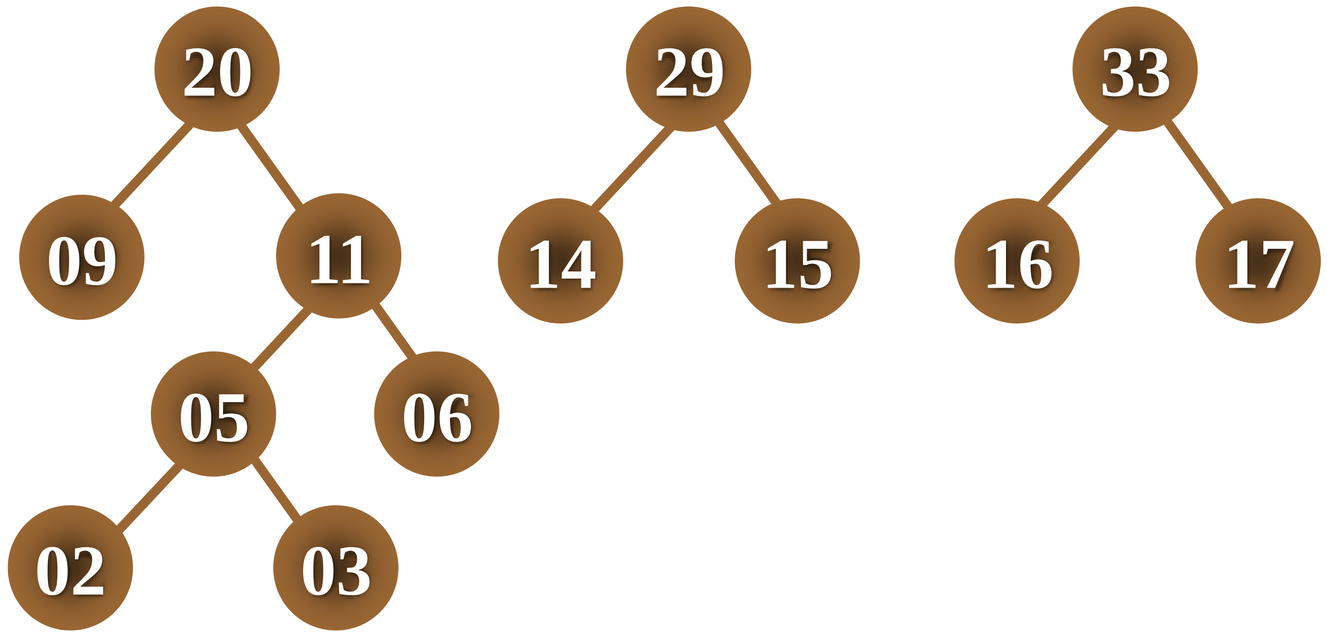
作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度



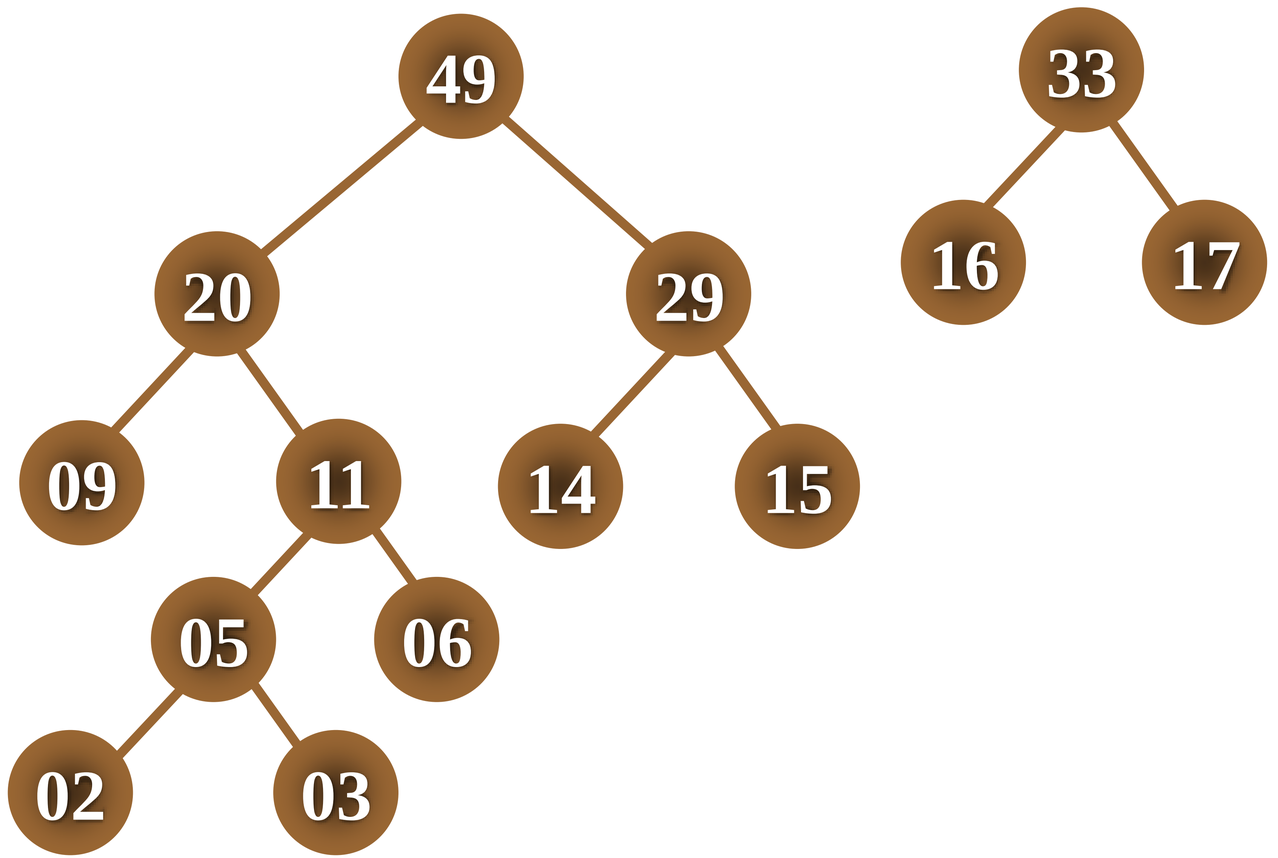
作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度



作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度



作业

- 给定权值集合{15, 03, 14, 02, 06, 09, 16, 17}， 构造相应的霍夫曼树，并计算它的带权路径长度

